

Programme de colle 4

Thibault Mabillotte

2 Questions de Cours exigibles

1. Connaître le vocabulaire lié aux couplages, savoir citer le théorème de Berge et savoir construire le graphe orienté permettant de trouver un chemin augmentant

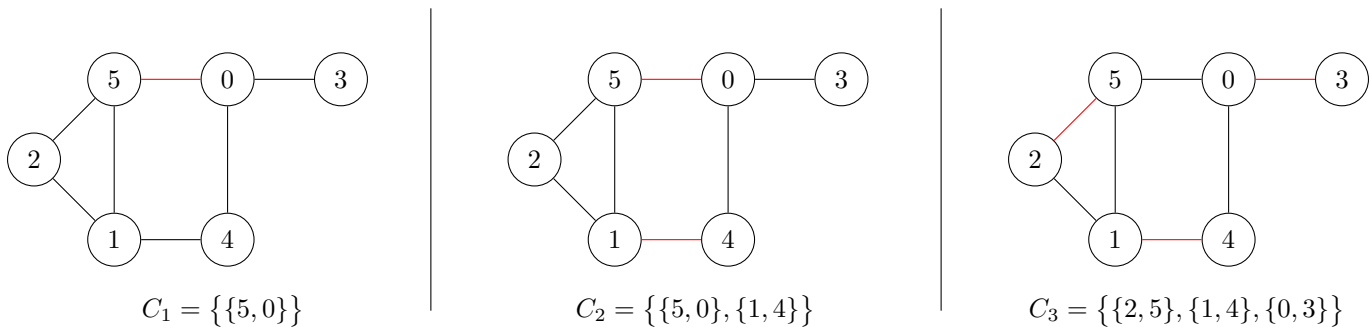
Définition : Soit $G = (S, A)$ un graphe orienté. On appelle couplage de G un sous-ensemble $C \subset A$ tel que pour tout $(a, b) \in C^2$, $a \neq b \implies a \cap b = \emptyset$. Autrement dit, un couplage est un sous ensemble des arêtes de G qui n'ont pas de sommets en commun.

Définition : Un couplage C de G est dit maximal si pour toute arête $a \notin C$, $C \cup \{a\}$ n'est pas un couplage pour G .

Définition : Un couplage C de G est dit maximum si son cardinal est le maximum parmi tous les couplages de G .

Définition : Soient $G = (S, A)$ un graphe et C un couplage de G . Un sommet $s \in S$ est dit apparié pour C s'il apparaît dans une arête de C . Dans le cas contraire, il est dit libre pour C .

Exemple On considère trois couplages représentés en rouge :

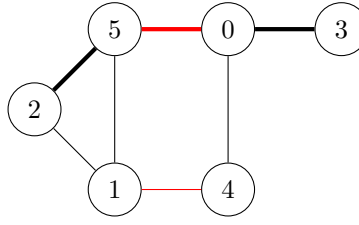


C_1 est un couplage, C_2 est un couplage maximal non maximum et C_3 est un couplage maximum.

Définition : Soient $G = (S, A)$ un graphe non orienté et C un couplage de G .

- On appelle chemin alternant pour C dans G un chemin élémentaire de longueur non-nulle dont les arêtes sont alternativement dans C et dans $A \setminus C$.
- On appelle chemin augmentant pour C dans G un chemin alternant qui commence et termine par des sommets libres.

Exemple : On considère le même graphe que précédemment. Le chemin représenté en gras est un chemin augmentant pour C_2 :



$$C_2 = \{\{5, 0\}, \{1, 4\}\}$$

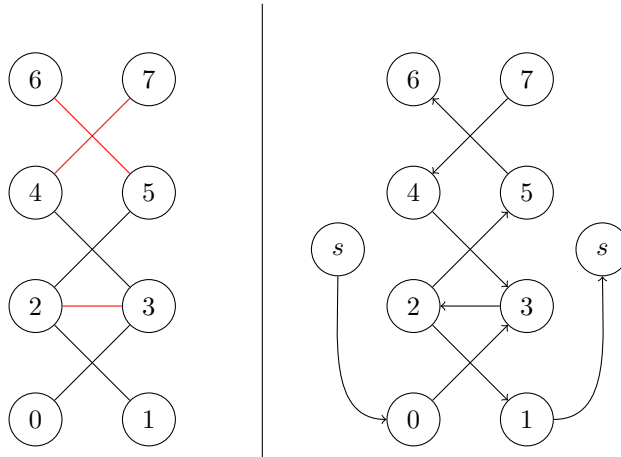
Remarque : Un chemin augmentant est toujours de taille impair $2k+1$ avec exactement $k+$ arêtes hors du couplage et k dans le couplage. Le chemin augmentant exhibé au dessus est de longueur 3 et possède deux arêtes hors de C_2 .

Théorème : Soient $G = (S, A)$ un graphe non orienté et C un couplage de G . Alors C est un couplage maximum de G si et seulement s'il n'existe pas de chemin augmentant pour C .

Définition : Soit $G = (S, A)$ un graphe. On dit que G est biparti s'il existe une partition de ses sommets en deux ensembles S_1 et S_2 telle que les arêtes de G ne sont qu'entre un sommet de S_1 et un sommet de S_2 .

Définition : Soit $G = (S, A)$ un graphe biparti avec $S = S_1 \sqcup S_2$. Soit C un couplage de G . On définit le graphe orienté associé à C par $G_C = (S', A')$ avec $S' = S \cup \{x, t\}$ et pour tout $x \in S_1$, $(s, x) \in A'$ si et seulement si x est libre pour C , pour tout $x \in S_2$, $(x, t) \in A'$ si et seulement si x est libre pour C et pour toute arête $\{u, v\} \in A$ avec $u \in S_1$ et $v \in S_2$, $(u, v) \in A'$ si $\{u, v\} \notin C$ et $(v, u) \in A'$ sinon.

Exemple : On considère un graphe biparti G et un couplage C dans G dont les arêtes sont représentées en rouge. Construisons le graphe G_C associé à C :



Théorème : Soient $G = (S, A)$ un graphe biparti, C un couplage et G_C le graphe associé à C . Alors G possède un chemin augmentant pour C si et seulement s'il existe un chemin de s à t dans G_C . S'il existe un tel chemin noté $(s, s_1, \dots, s_k, t)$ alors $(s_1, \dots, s_k)$ est un chemin augmentant pour C .

2. Savoir appliqué l'algorithme de Berry Sethi

L'objectif de l'algorithme de Berry Sethi est de transformer une expression régulière en automate. Ainsi, à partir d'une expression régulière e , l'algorithme construit l'automate de Glushkov $\mathcal{A}$ tel que $\mathcal{L}(e) = \mathcal{L}(\mathcal{A})$.

Ainsi, le langage reconnu par une expression régulière est reconnaissable. Cela implique que la classe des langages réguliers est incluse dans celle des langages reconnaissables.

Le linéarisé d'une expression régulière

Définition : Une expression régulière est linéaire si dans son écriture chaque lettre de l'alphabet n'apparaît qu'une seule fois.

Définition : Soit e une expression régulière quelconque. On appelle linéarisé de e la même expression e' dans laquelle on duplique chaque occurrence d'une même lettre en des lettres différentes.

Exemple : Si $e = a^*(ab)^*$ alors le linéarisé de e est $e' = a_1^*(a_2b)^*$.

Les langages locaux

Définition : Un langage L sur l'alphabet Σ est local s'il existe $P \subset \Sigma, S \subset \Sigma$ et $N \subset \Sigma^2$ tels que :

$$L \setminus \{\epsilon\} = (P\Sigma^* \cap \Sigma^*S) \setminus (\Sigma^*N\Sigma^*)$$

Intuitivement, P représente l'ensemble des lettres par lesquelles peut commencer un mot de L . S représente l'ensemble des lettres par lesquelles peut terminer un mot de L . N est l'ensemble des facteurs de taille 2 qui ne sont pas présents dans un mot de L . En pratique, on pose $F = \Sigma^2 \setminus N$.

Exemple : Le langage décrit par $e = aa^*$ est local car tous les mots de ce langage commencent par a , finissent par a et la lettre a ne peut être suivie que d'un autre a .

En revanche, le langage décrit par $e = a + (ab)^*$ n'est pas local. S'il l'était alors on aurait $P = \{a\}$, $S = \{a, b\}$ et $F = \{ab, ba\}$. $aba \notin \mathcal{L}(e)$. Pourtant, aba commence par $a \in P$, fini par un $a \in S$ et ses facteurs de taille 2 sont bien dans F .

Théorème : Tout langage local est reconnaissable.

Démonstration : Notons L un langage local sur l'alphabet Σ . On va faire une preuve constructive de ce théorème, c'est-à-dire qu'on va montrer qu'il existe un automate $\mathcal{A}$ qui reconnaît L en construisant $\mathcal{A}$.

Ainsi, pour construire un automate qui reconnaît un langage local particulier, il suffit d'appliquer cette méthode à ce cas particulier.

L est local donc on peut lui associer P , S et F définis plus haut. On définit alors l'automate $\mathcal{A}$ suivant :

$$\mathcal{A} = \left(\{q_0\} \cup \{q_\alpha \mid \alpha \in \Sigma\}, q_0, \delta, F' \right)$$

Intuitivement, arriver sur l'état q_α signifie que la lettre α vient d'être lue. On définit ensuite la fonction de transition :

$$\delta : \begin{cases} \forall \alpha \in P & \delta(q_0, \alpha) = q_\alpha \\ \forall (\alpha, \beta) \in F & \delta(q_\alpha, \beta) = q_\beta \end{cases}$$

Ainsi, on peut passer de l'état q_α à l'état q_β si et seulement si $\alpha\beta$ est un facteur autorisé. Dans le cas contraire, l'automate bloque et le mot n'est pas reconnu. On définit enfin F' par :

$$F' = \{q_\alpha \mid \alpha \in S\}$$

Ici, un mot n'est accepté par l'automate $\mathcal{A}$ que s'il termine par une lettre de S .

Par construction, $\mathcal{A}$ accepte exactement les mots de longueur au moins 1 qui commencent par une lettre de P , qui terminent par une lettre de S et qui ne contiennent que des facteurs de taille 2 appartenant à F .

Si L reconnaît le mot vide, on ajoute q_0 à l'ensemble des états finaux de $\mathcal{A}$.

On trouve donc bien que $\mathcal{A}$ reconnaît L . Ainsi L est reconnaissable.

Théorème (admis) : Un langage dénoté par une expression linéaire est local.

Automate de Glushkov

On veut construire, à partir d'une expression régulière quelconque e , un automate $\mathcal{A}$ qui reconnaît le langage dénoté par e .

On sait que e peut être linéarisée en une expression linéaire e' . De plus, pour toute expression linéaire, on sait construire un automate qui la reconnaît.

Ainsi, pour construire l'automate de Glushkov associé à e on suit la méthode suivante :

1. On linéarise e .
2. On construit l'automate qui reconnaît le linéarisé de e en utilisant la méthode vue plus haut.
3. On transforme cet automate en un automate qui reconnaît e .

Exemple

Exemple : On pose $e = a(a + ba^*b)^*ba^*$. Construisons l'automate de Glushkov qui reconnaît $L = \mathcal{L}(e)$.

On commence par linéariser e . On trouve $e' = a_1(a_2 + b_1a_3^*b_2)^*b_3a_4^*$.

Ensuite, il faut trouver P , S et F . Ici, $P = \{a_1\}$ car un mot dénoté par e' commence obligatoirement par a_1 . En revanche, un mot dénoté par e' peut terminer par a ou b donc $S = \{a_4, b_3\}$.

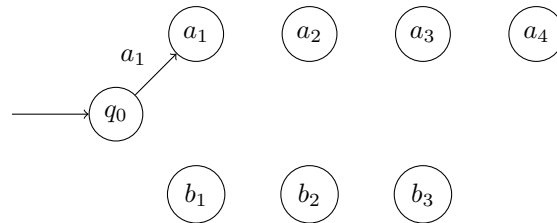
Pour trouver les facteurs, il ne faut faire attention au étoile. Ici, b_2 peut être suivi de b_3 mais aussi de a_2 et b_1 . Il est conseillé de trouver les facteurs en suivant une progression linéaire dans e' . On commence par toutes les lettres qui suivent a_1 . Puis celles qui suivent a_2 etc etc.

On trouve finalement $F = \{a_1a_2, a_1b_1, a_1b_3, a_2a_2, a_2b_1, a_2b_3, b_1a_3, b_1b_2, a_3a_3, a_3b_2, b_2a_2, b_2b_1, b_2b_3, b_3a_4, a_4a_4\}$.

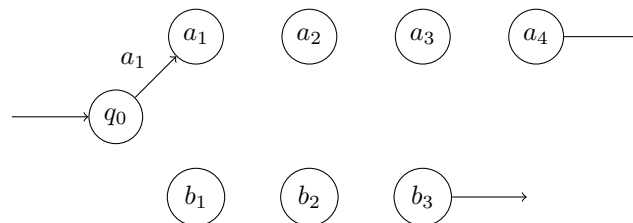
On construit maintenant l'automate reconnaissant e' à l'aide de la méthode vu plus haut.

D'abord on ajoute l'état initial et, pour chaque lettre α de l'expression e' , l'état q_α . On le note simplement α .

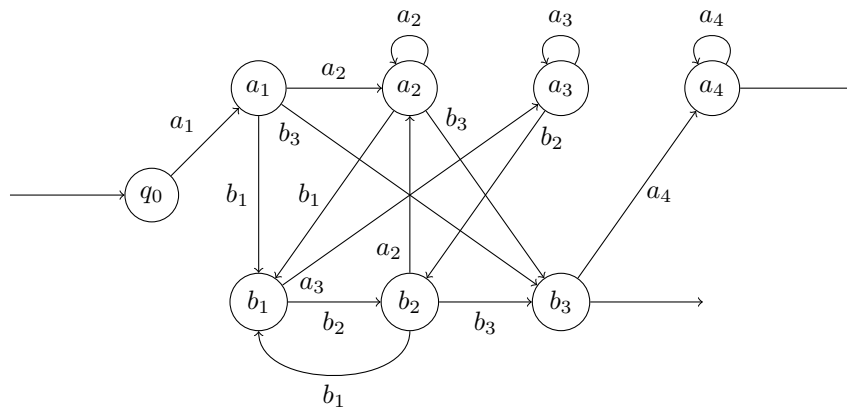
Pour chaque lettre α dans P , on ajoute une transition étiquetée α entre q_0 et l'état q_α .



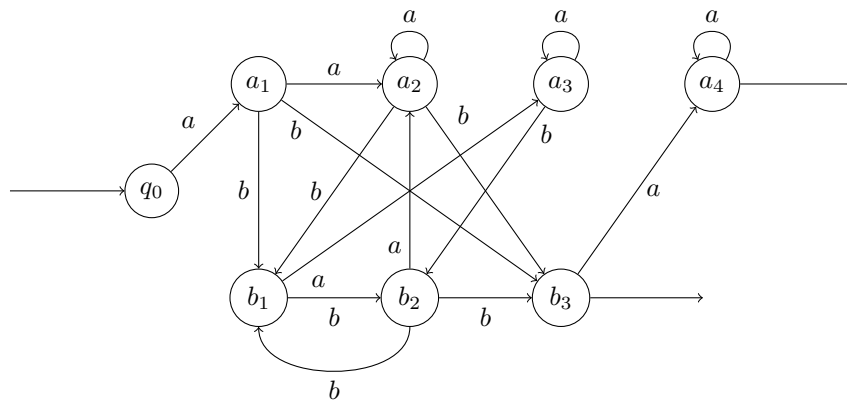
Ensuite, on transforme les lettres appartenant à S en état final. Par ailleurs, le mot vide n'appartient pas à L donc q_0 n'est pas un état final. On a :



Enfin, pour chaque $\alpha\beta \in F$, on crée une transition étiquetée par β entre l'état q_α et l'état q_β . Par exemple, $a_1a_2 \in F$ donc on ajoute une transition étiquetée par a_2 entre a_1 et a_2 . Finalement on obtient :



Ainsi, on a construit un automate qui reconnaît le langage dénoté par e' . Pour obtenir l'automate de Glushkov qui reconnaît le langage dénoté par e , il suffit de "retirer les indices" des lettres. On trouve donc :



3. Savoir appliquer la méthode d'élimination des états

Méthode générale de l'élimination des états

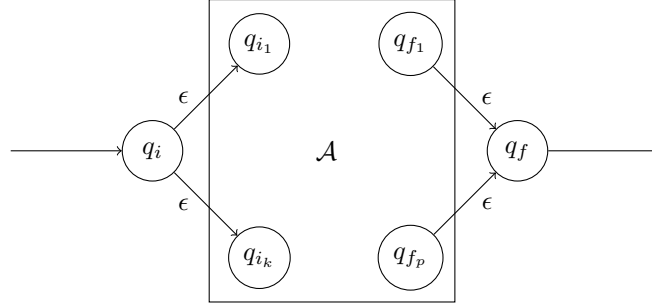
L'objectif de la méthode d'élimination des états est de transformer un automate en expression régulière. Ainsi, à partir d'un automate $\mathcal{A}$ quelconque, cette méthode construit une expression régulière e telle que $\mathcal{L}(\mathcal{A}) = \mathcal{L}(e)$.

La méthode se décompose en deux étapes. On note $\mathcal{A} = (Q, I, F, \delta, \varphi)$.

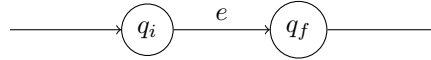
D'une part, on standardise l'automate $\mathcal{A}$, c'est-à-dire qu'on transforme $\mathcal{A}$ en automate avec un unique état initial sans transition entrante et un unique état final sans transition sortante.

Pour ce faire, on ajoute un état q_i , qui sera l'unique état initial, et on le relie avec des ϵ -transitions aux anciens états initiaux. De même, on ajoute un état final q_f relié aux anciens états finaux.

On obtient un automate de la forme :



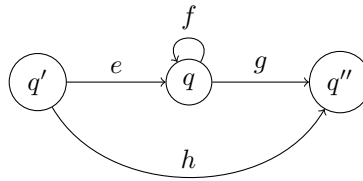
D'autre part, on élimine un à un les états jusqu'à ce qu'il ne reste plus que l'état initial et final. On trouve un automate de la forme :



Pour éliminer un état $q \in Q$, on utilise la méthode suivante :

On pose $E_q = \{q' \in Q \setminus \{q\} \mid \exists e, \delta(q', e) = q\}$ et $S_q = \{q'' \in Q \setminus \{q\} \mid \exists e, \delta(q, e) = q''\}$. Autrement dit, l'ensemble des états précédant q et succédant à q dans $\mathcal{A}$.

Pour chaque couple $(q', q'') \in E_q \times S_q$ on considère les différentes transitions entre q , q' et q'' :

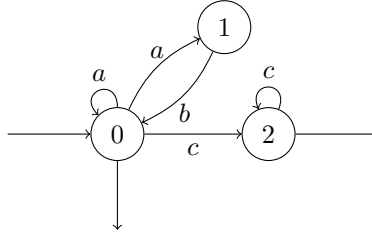


On ajoute une transition entre q' et q'' étiquetée par l'expression $ef^*g|h$ (ou on la remplace si elle existe déjà). Une fois que tous les couples (q', q'') ont été traités, on supprime l'état q .

Finalement, la transition entre l'état initial et l'état final est étiquetée par une expression régulière qui dénote le langage reconnu par l'automate.

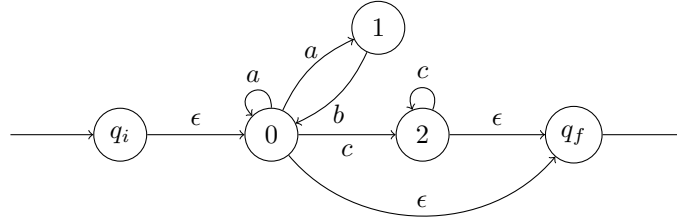
Exemple

On considère l'automate suivant :



On se propose de trouver une expression régulière qui dénote le langage reconnu par cet automate en utilisant la méthode d'élimination des états. On impose d'éliminer les états dans l'ordre suivant : 2, 1 puis 0.

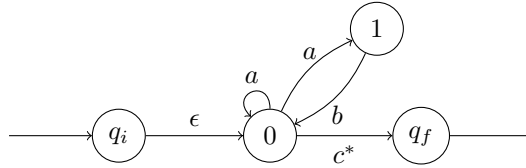
On commence par standardiser l'automate. On trouve :



Il faut maintenant éliminer les états en commençant par l'état 2.

Tout d'abord, on trouve E_2 et S_2 . Ici $E_2 = \{0\}$ et $S_2 = \{q_f\}$. On doit donc ajouter une transition entre l'état 0 et l'état q_f (ou la modifier si elle existe déjà). Avec les mêmes notations que plus haut, on a $e = c$, $f = c$, $g = \epsilon$ et $h = \epsilon$. On étiquette la transition par $cc^*\epsilon|\epsilon$. On remarque que cette expression régulière dénote le même langage que c^* .

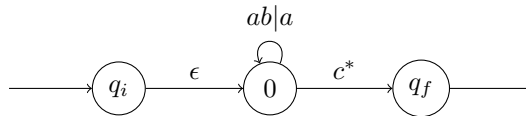
Une fois cette transition ajoutée, il ne reste plus de couple (q', q'') à traiter donc on élimine l'état 2 :



On élimine à présent l'état 1.

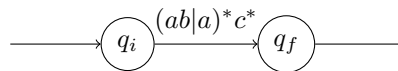
On trouve $E_1 = \{0\}$ et $S_1 = \{0\}$. On doit donc ajouter une transition entre 0 et 0. Ici, on doit la modifier car elle existe déjà. On a $e = a$, $f = \epsilon$, $g = b$ et $h = a$. On remarque qu'il n'y a pas de boucle sur 1 donc on pose $f = \epsilon$. De plus, l'état précédant 1 et succédant à 1 est le même, ici 0. Cela ne pose pas de problème, on va juste ajouter une boucle sur 0 étiquetée par $ef^*g|h$.

Ceci étant fait, il n'y a plus de couple (q', q'') à traiter donc on élimine l'état 1 :



Il ne reste qu'à éliminer l'état 0.

On a $E_0 = \{q_i\}$ et $S_0 = \{q_f\}$. On a $e = \epsilon$, $f = ab|a$, $g = c^*$ et $h = \epsilon$. Ici, il n'y a pas de transition entre q_i et q_f donc on laisse $h = \epsilon$. On élimine 0 et on trouve :



Finalement, on trouve $e = (ab|a)^*c^*$ une expression régulière qui dénote le langage reconnu par l'automate.

4. Savoir exhiber un arbre de preuve en déduction naturelle

Conseil : Pour démontrer un séquent, il est essentiel d'appliquer les règles de la déduction naturelle enseignées en cours. La première étape consiste donc à mémoriser ces règles. Bien qu'elles soient rappelées lors des concours, Madame Monfleur exige qu'elles soient parfaitement connues lors des colles. Par ailleurs, maîtriser les règles de dérivation facilite la compréhension de la structure des arbres logiques et permet de développer une meilleure intuition.

Il est également important de noter que certaines règles s'appliquent presque systématiquement. Par exemple, pour démontrer une implication, on utilise généralement la règle d'introduction de l'implication. De même, pour établir la négation d'une formule, on passe par l'introduction de la négation, suivie de son élimination.

Voici des exemples d'arbre de preuve en déduction naturelle :

Exemple : Montrons le séquent $\varphi \rightarrow \neg\varphi \vdash \neg\varphi$.

$$\frac{\frac{\frac{}{\varphi \rightarrow \neg\varphi, \varphi \vdash \varphi} (r) \quad \frac{\frac{}{\varphi \rightarrow \neg\varphi, \varphi \vdash \varphi \rightarrow \neg\varphi} (r) \quad \frac{}{\varphi \rightarrow \neg\varphi, \varphi \vdash \varphi} (E\rightarrow)}{\varphi \rightarrow \neg\varphi, \varphi \vdash \neg\varphi} (E\rightarrow)}{\varphi \rightarrow \neg\varphi, \varphi \vdash \square} (I\rightarrow) \quad \frac{}{\varphi \rightarrow \neg\varphi \vdash \neg\varphi} (I\rightarrow)$$

Exemple : Montrons le séquent $\neg\varphi \vee \psi \vdash \varphi \rightarrow \psi$.

$$\frac{\frac{\frac{}{\neg\varphi \vee \psi, \varphi \vdash \neg\varphi \vee \psi} (r) \quad \frac{\frac{}{\neg\varphi \vee \psi, \varphi \vdash \neg\varphi} (r) \quad \frac{}{\neg\varphi \vee \psi, \varphi \vdash \psi} (E\rightarrow)}{\neg\varphi \vee \psi, \varphi \vdash \psi} (I\rightarrow)}{\neg\varphi \vee \psi \vdash \varphi \rightarrow \psi} (I\rightarrow)$$

Exemple : Montrons le séquent $\varphi \rightarrow (\psi \rightarrow \theta), \psi \rightarrow \varphi \vdash \psi \rightarrow \theta$.

$$\frac{\frac{\frac{}{\varphi \rightarrow (\psi \rightarrow \theta) \vdash \varphi \rightarrow (\psi \rightarrow \theta)} (r) \quad \frac{\frac{}{\psi \rightarrow \varphi \vdash \psi \rightarrow \varphi} (r) \quad \frac{}{\Gamma, \psi \vdash \psi \rightarrow \varphi} (m)}{\Gamma, \psi \vdash \varphi} (E\rightarrow)}{\Gamma, \psi \vdash \psi \rightarrow \theta} (I\rightarrow) \quad \frac{}{\Gamma, \psi \vdash \psi} (r) \quad \frac{}{\Gamma, \psi \vdash \theta} (E\rightarrow) \quad \frac{}{\Gamma := \varphi \rightarrow (\psi \rightarrow \theta), \psi \rightarrow \varphi \vdash \psi \rightarrow \theta} (I\rightarrow)$$

Exemple : Montrons le séquent $(\varphi_1 \wedge \varphi_2) \rightarrow \psi \vdash \varphi_1 \rightarrow (\varphi_2 \rightarrow \psi)$.

$$\frac{\frac{\frac{}{(\varphi_1 \wedge \varphi_2) \rightarrow \psi \vdash (\varphi_1 \wedge \varphi_2) \rightarrow \psi} (r) \quad \frac{\frac{}{\Gamma, \varphi_1, \varphi_2 \vdash \varphi_1} (r) \quad \frac{}{\Gamma, \varphi_1, \varphi_2 \vdash \varphi_2} (r)}{\Gamma, \varphi_1, \varphi_2 \vdash \varphi_1 \wedge \varphi_2} (E\wedge)}{\Gamma, \varphi_1, \varphi_2 \vdash \psi} (I\rightarrow) \quad \frac{}{\Gamma, \varphi_1 \vdash \varphi_2 \rightarrow \psi} (I\rightarrow) \quad \frac{}{\Gamma := (\varphi_1 \wedge \varphi_2) \rightarrow \psi \vdash \varphi_1 \rightarrow (\varphi_2 \rightarrow \psi)} (I\rightarrow)$$

Exemple : Montrons le séquent $\varphi_1 \rightarrow (\varphi_2 \rightarrow \psi) \vdash (\varphi_1 \wedge \varphi_2) \rightarrow \psi$.

$$\frac{\frac{\frac{}{\varphi_1 \rightarrow (\varphi_2 \rightarrow \psi) \vdash \varphi_1 \rightarrow (\varphi_2 \rightarrow \psi)} (r) \quad \frac{\frac{}{\Gamma, \varphi_1 \wedge \varphi_2 \vdash \varphi_1 \wedge \varphi_2} (r) \quad \frac{}{\Gamma, \varphi_1 \wedge \varphi_2 \vdash \varphi_1} (E\wedge)}{\Gamma, \varphi_1 \wedge \varphi_2 \vdash \varphi_2 \rightarrow \psi} (E\rightarrow) \quad \frac{}{\Gamma, \varphi_1 \wedge \varphi_2 \vdash \varphi_2} (E\wedge)}{\Gamma, \varphi_1 \wedge \varphi_2 \vdash \psi} (I\rightarrow) \quad \frac{}{\Gamma := \varphi_1 \rightarrow (\varphi_2 \rightarrow \psi) \vdash (\varphi_1 \wedge \varphi_2) \rightarrow \psi} (I\rightarrow)$$

5. Savoir prouver le théorème de correction de la déduction naturelle pour la sémantique standard et savoir l'utiliser pour justifier qu'un séquent n'est pas dérivable

Théorème

Théorème (de correction) : Soient Γ un ensemble de formules et ϕ une formule. Alors :

$$\Gamma \vdash \phi \implies \Gamma \models \phi$$

Remarque : Jusqu'ici, dans le cours, $\vdash$ est un système de déduction formelle qui n'a pas de sens en soi. On dit simplement que ϕ se déduit de Γ si $\Gamma \vdash \phi$ et on détermine quand c'est le cas grâce aux règles de déduction naturelle que l'on s'est donné. Ce théorème permet donc de donner un sens aux séquents : si ϕ se déduit de Γ alors ϕ est une conséquence sémantique de Γ .

Démonstration : Montrons ce résultat par récurrence sur la hauteur de l'arbre de dérivation.

Posons H_n : "Pour tout ensemble de formules Γ et toute formule ϕ tels qu'il existe un arbre de dérivation du séquent $\Gamma \vdash \phi$ de taille n , on a $\Gamma \models \phi$."

Pour $n = 0$. Soient Γ et ϕ tel que le séquent $\Gamma \vdash \phi$ soit dérivable en un arbre de hauteur 0. L'arbre étant de hauteur 0, il est forcément de la forme :

$$\frac{}{\Gamma \vdash \phi} (r)$$

Ainsi, par définition de la réflexivité, $\phi \in \Gamma$. On a donc nécessairement $\Gamma \models \phi$.

Soit $n \in \mathbb{N}$. Supposons $H_0, \dots, H_n$ vraies et montrons H_{n+1} .

Soient Γ et ϕ tel que le séquent $\Gamma \vdash \phi$ soit dérivable en un arbre de hauteur $n + 1$. Il y a alors 8 cas possibles selon la dernière règle utilisée dans l'arbre de dérivation.

Supposons que la règle utilisée soit l'élimination de la négation. On a alors :

$$\frac{\frac{\dots}{\Gamma \vdash \psi} \quad \frac{\dots}{\Gamma \vdash \neg \psi}}{\Gamma \vdash \phi} (E\neg)$$

L'arbre de dérivation du séquent $\Gamma \vdash \phi$ se divise en deux sous arbres de taille inférieure ou égale à n . Ainsi, l'hypothèse de récurrence forte s'applique aux deux arbres. On a donc que $\Gamma \models \psi$ et $\Gamma \models \neg \psi$.

Autrement dit, tout modèle de Γ est un modèle de ψ et est aussi un modèles de $\neg \psi$. On trouve donc que l'ensemble des modèles de Γ est vide. Dès lors, on peut affirmer que tout modèle de Γ est un modèle de ϕ . D'où $\Gamma \models \phi$.

Supposons que la règle utilisé soit l'élimination de $\wedge$. On a alors :

$$\frac{\frac{\dots}{\Gamma \vdash \phi \wedge \psi}}{\Gamma \vdash \phi} (E\wedge)$$

L'arbre de dérivation se divise en un seul sous arbre de taille n . Par hypothèse de récurrence, on a que $\Gamma \models \phi \wedge \psi$.

Ainsi, tout modèle de Γ est un modèle de $\phi \wedge \psi$ donc, d'après la sémantique standard, un modèle de ϕ . D'où $\Gamma \models \phi$.

Supposons que la règle utilisé soit l'introduction de l'implication. On pose $\phi = \chi \rightarrow \psi$ et on a :

$$\frac{\frac{\dots}{\Gamma, \chi \vdash \psi}}{\Gamma \vdash \chi \rightarrow \psi} (I\rightarrow)$$

De même, par hypothèse de récurrence, $\Gamma, \chi \models \psi$. C'est-à-dire que tout modèle σ de Γ et χ est un modèle de ψ . Montrons que tout modèle de Γ est un modèle de $\chi \rightarrow \psi$.

Soit σ un modèle de Γ . Montrons que σ est un modèle de $\chi \rightarrow \psi$. Si $\sigma \models \chi$ alors par hypothèse $\sigma \models \psi$ et donc $\sigma \models \chi \rightarrow \psi$ d'après la sémantique standard. Si $\sigma \not\models \chi$, par la sémantique standard on sait que $\sigma \models \chi \rightarrow \psi$.

Ainsi, tout modèle de Γ est aussi un modèle de $\chi \rightarrow \psi$, d'où $\Gamma \models \chi \rightarrow \psi$.

Supposons que la règle utilisé soit l'élimination du $\vee$. On a alors :

$$\frac{\frac{\dots}{\Gamma \vdash \chi \vee \psi} \quad \frac{\dots}{\Gamma, \chi \vdash \phi} \quad \frac{\dots}{\Gamma, \psi \vdash \phi}}{\Gamma \vdash \phi} (E\vee)$$

L'arbre de dérivation du séquent $\Gamma \vdash \phi$ se divise en trois sous arbres de taille inférieure ou égale à n . Ainsi, l'hypothèse de récurrence forte s'applique aux trois arbres. On a donc que $\Gamma \models \chi \vee \psi$, $\Gamma, \chi \models \phi$ et $\Gamma, \psi \models \phi$.

Tout modèle de Γ est un modèle de $\chi \vee \psi$, tout modèle de Γ et χ est un modèle de ϕ et tout modèle de Γ et ψ est un modèle de ϕ .

Soit σ un modèle de Γ . Montrons que σ est un modèle ϕ . Comme $\sigma \models \Gamma$ on sait que $\sigma \models \chi \vee \psi$. Par la sémantique standard, on a que σ est un modèle de χ ou de ψ . Supposons, sans perte de généralité, que σ soit un modèle de ψ . On a alors que $\sigma \models \Gamma, \psi$ donc $\sigma \models \phi$.

Ainsi, tout modèle de Γ est aussi un modèle de ϕ , d'où $\Gamma \models \phi$.

En faisant la même chose pour les autres règles, on trouve que pour tout ensemble de formules Γ et toute formule ϕ tels que le séquent $\Gamma \vdash \phi$ soit dérivable en un arbre de hauteur $n + 1$, on a $\Gamma \models \phi$. Ainsi, H_{n+1} est vraie.

Par principe de récurrence, le théorème de correction est vrai.

Utilisation du théorème

Le théorème de correction peut être utile pour montrer qu'un séquent n'est pas prouvable. En effet, par contraposée du théorème, si $\Gamma \not\models \phi$ alors $\Gamma \not\vdash \phi$.

Exemple : Montrons que $\vdash \phi \wedge \neg\phi$ n'est pas prouvable.

En sémantique standard, ϕ et $\neg\phi$ ne peuvent pas être vrai simultanément donc l'ensemble des modèles de $\phi \wedge \neg\phi$ est vide. En particulier, $\not\models \phi \wedge \neg\phi$.

Ainsi, par contraposée du théorème de correction, le séquent $\vdash \phi \wedge \neg\phi$ n'est pas prouvable.

6. Connaitre deux preuves du fait que les langages réguliers sont non contextuels

On veut montrer que la classe des langages réguliers (*Reg*) est incluse dans la classe des langages non contextuels (*CFL*). Or on sait, d'après la théorème de Kleene, que la classe des langages réguliers est exactement la classe des langages reconnaissables (*Rec*). Ainsi, on peut aussi montrer que les langages reconnaissables sont non contextuels.

Le programme impose de connaître les deux méthodes.

Commençons par montrer que les langages réguliers sont non contextuels en utilisant la définition inductive des langages réguliers. Soit Σ un alphabet.

On rappelle la définition de la classe des langages réguliers sur l'alphabet Σ :

- $\emptyset$ est un langage régulier.
- $\forall a \in \Sigma, \{a\}$ est un langage régulier.
- Si L_1 et L_2 sont des langages réguliers alors $L_1 \cup L_2$ est un langage régulier.
- Si L_1 et L_2 sont des langages réguliers alors $L_1 L_2$ est un langage régulier.
- Si L est un langage régulier alors L^* est un langage régulier.

Pour montrer que $Reg \subset CFL$, il suffit de montrer que $\emptyset \in CFL$, que $\forall a \in \Sigma, \{a\} \in CFL$ et que *CFL* est stable par union, concaténation et étoile de Kleene.

On pose la grammaire $\mathcal{G}_\emptyset = (\{S\}, \Sigma, S, R_\emptyset)$ où $R_\emptyset$ est l'ensemble des règles de dérivation qui ne contient que $S \rightarrow S$.

On remarque que la seule règle de $\mathcal{G}_\emptyset$ ne permet pas d'aboutir à un mot ne contenant que des symboles non terminaux. Autrement dit, cette grammaire ne génère aucun mot donc $\mathcal{L}(\mathcal{G}_\emptyset) = \emptyset$.

On a bien $\emptyset \in CFL$.

Soit $a \in \Sigma$. On pose $\mathcal{G}_a = (\{S\}, \Sigma, S, R_a)$ où R_a contient l'unique règle $S \rightarrow a$.

On remarque que la seul mot généré par $\mathcal{G}_a$ est a donc $\mathcal{L}(\mathcal{G}_a) = \{a\}$.

Ainsi, $\forall a \in \Sigma, \{a\} \in CFL$.

Soient L_1 et L_2 deux langages non contextuels. Il existe $\mathcal{G}_1 = (\mathcal{V}_1, \Sigma, S_1, R_1)$ et $\mathcal{G}_2 = (\mathcal{V}_2, \Sigma, S_2, R_2)$ telles que $L_1 = \mathcal{L}(\mathcal{G}_1)$ et $L_2 = \mathcal{L}(\mathcal{G}_2)$.

On pose $\mathcal{G}_\cup = (\mathcal{V}_1 \cup \mathcal{V}_2 \cup \{S_{\text{new}}\}, \Sigma, S_{\text{new}}, R_\cup)$ avec $S_{\text{new}} \notin \mathcal{V}_1 \cup \mathcal{V}_2$ et $R_\cup = \{S_{\text{new}} \rightarrow S_1, S_{\text{new}} \rightarrow S_2\} \cup R_1 \cup R_2$.

On suppose que $\mathcal{V}_1$ et $\mathcal{V}_2$ sont disjoints. Cette hypothèse est importante est garantie que les règles de $\mathcal{G}_1$ ne vont pas interférer avec les règles de $\mathcal{G}_2$. Ainsi, la grammaire $\mathcal{G}_\cup$ n'engendre que des mots de $L_1 \cup L_2$.

Avec cette hypothèse, on montre très facilement que $\mathcal{L}(\mathcal{G}_\cup) = L_1 \cup L_2$. Donc $L_1 \cup L_2$ est non contextuel.

La classe *CFL* est stable par union.

On garde les mêmes langages L_1 et L_2 . On pose $\mathcal{G}_\bullet = (\mathcal{V}_1 \cup \mathcal{V}_2 \cup \{S_{\text{new}}\}, \Sigma, S_{\text{new}}, R_\bullet)$ avec $S_{\text{new}} \notin \mathcal{V}_1 \cup \mathcal{V}_2$ et $R_\bullet = \{S_{\text{new}} \rightarrow S_1 S_2\} \cup R_1 \cup R_2$.

Avec la même hypothèse de disjonction entre $\mathcal{V}_1$ et $\mathcal{V}_2$, on montre que $\mathcal{L}(\mathcal{G}_\bullet) = L_1 L_2$. Donc $L_1 L_2$ est non contextuel.

La classe *CFL* est stable par concaténation.

Soit L un langage non contextuel. Il existe $\mathcal{G} = (\mathcal{V}, \Sigma, S, R)$ telle que $L = \mathcal{L}(\mathcal{G})$.

On pose $\mathcal{G}_* = (\mathcal{V} \cup \{S_{\text{new}}\}, \Sigma, S_{\text{new}}, R_*)$ avec $S_{\text{new}} \notin \mathcal{V}$ et $R_* = \{S_{\text{new}} \rightarrow S S_{\text{new}}, S_{\text{new}} \rightarrow \epsilon\} \cup R$.

On montre que $\mathcal{L}(\mathcal{G}_*) = L^*$. Ainsi, L^* est non contextuel.

La classe *CFL* est stable par étoile de Kleene.

D'après la définition inductive, un langage régulier est formé par des unions, des concaténations et des itérations (étoiles de Kleene) à partir du langage vide et des singletons. Or, ces derniers sont non contextuels, et la classe des langages non contextuels est stable par union, concaténation et étoile de Kleene. Par conséquent, tout langage régulier est nécessairement un langage non contextuel.

On a bien $Reg \subset CFL$.

À présent, montrons que les langages reconnaissables sont non contextuels.

Soit L un langage reconnaissable. Il existe donc $\mathcal{A} = (\Sigma, Q, q_0, F, \delta)$ tel que $L = \mathcal{L}(\mathcal{A})$.

On pose $\mathcal{G} = (\{X_q \mid q \in Q\}, \Sigma, X_{q_0}, R)$ où R est défini par :

$$R = \{X_{q'} \rightarrow \alpha X_{q''} \mid \delta(q', \alpha) = q''\} \cup \{X_{q_f} \rightarrow \epsilon \mid q_f \in F\}$$

Montrons que pour tout mot $m \in \Sigma^*$ et tous états $q, q' \in Q$, on a :

$$\delta^*(q, m) = q' \iff X_q \Rightarrow^* m X_{q'}$$

Montrons ce résultat par récurrence sur la taille du mot m .

Posons $H_n : \forall q, q' \in Q, \forall m \in \Sigma^*, |m| = n, \delta(q, m) = q' \iff X_q \Rightarrow^* m X_{q'}$.

Pour $n = 0$. Soient $q, q' \in Q$ et $m \in \Sigma^*$ tel $|m| = 0$. On a donc $m = \epsilon$. Ainsi, $\delta^*(q, \epsilon) = q'$ si et seulement si $q = q'$. Il est immédiat que $X_q \Rightarrow^* X_{q'}$. H_0 est vraie.

Soit $n \in \mathbb{N}$. Supposons H_n vrai et montrons H_{n+1} .

Soient $q, q' \in Q$ et $m \in \Sigma^*$ tel que $|m| = n + 1$. Supposons que $\delta^*(q, m) = q'$. $n + 1 \geq 1$ donc il existe $a \in \Sigma$ et $m' \in \Sigma^*$ tels que $|m'| = n$ et $m = am'$. On trouve alors :

$$\delta^*(q, m) = q' \iff \delta^*(q, am') = q' \iff \delta^*(\delta(q, a), m') = q'$$

$|m'| = n$ donc par hypothèse de récurrence, on sait que :

$$X_{\delta(q, a)} \Rightarrow^* m' X_{q'}$$

De plus, par construction de la grammaire, on sait que $X_q \Rightarrow a X_{\delta(q, a)}$ donc :

$$X_q \Rightarrow a X_{\delta(q, a)} \Rightarrow^* am' X_{q'}$$

On a bien $X_q \Rightarrow^* m X_{q'}$. H_{n+1} est vraie.

Par principe de récurrence, la propriété est vraie pour tout mot $m \in \Sigma^*$.

On conclut alors :

$$m \in \mathcal{L}(\mathcal{A}) \iff \exists q_f \in F, \delta^*(q_0, m) = q_f \iff \exists q_f \in F, X_{q_0} \Rightarrow^* m X_{q_f}$$

Or, par construction de la grammaire, seuls les X_{q_f} avec $q_f \in F$ peuvent dériver ϵ . D'où la dernière équivalence :

$$m \in \mathcal{L}(\mathcal{A}) \iff \exists q_f, \delta^*(q_0, m) = q_f \iff X_{q_0} \Rightarrow^* m \iff m \in \mathcal{L}(\mathcal{G})$$

Ainsi, L est non contextuel. On a bien $Rec \subset CFL$.